CampiFi — Object-Oriented Programming (OOP) Implementation Guide

Demonstrating Classes, Objects, Encapsulation, Inheritance, Polymorphism, Abstraction & Interfaces in Java 17 & Spring Boot

---

Object-Oriented Programming (OOP) is the architectural foundation of **CampiFi**. Rather than treating campus chat as procedural script routines, CampiFi models the entire offline system around real-world domain abstractions: *Users*, *Channels*, *SubChannels*, *Messages*, and *Sessions*. This guide demonstrates how every core OOP principle is strictly implemented across the codebase, providing concrete source code mappings, theoretical justifications, and examiner-ready answers for project evaluation.

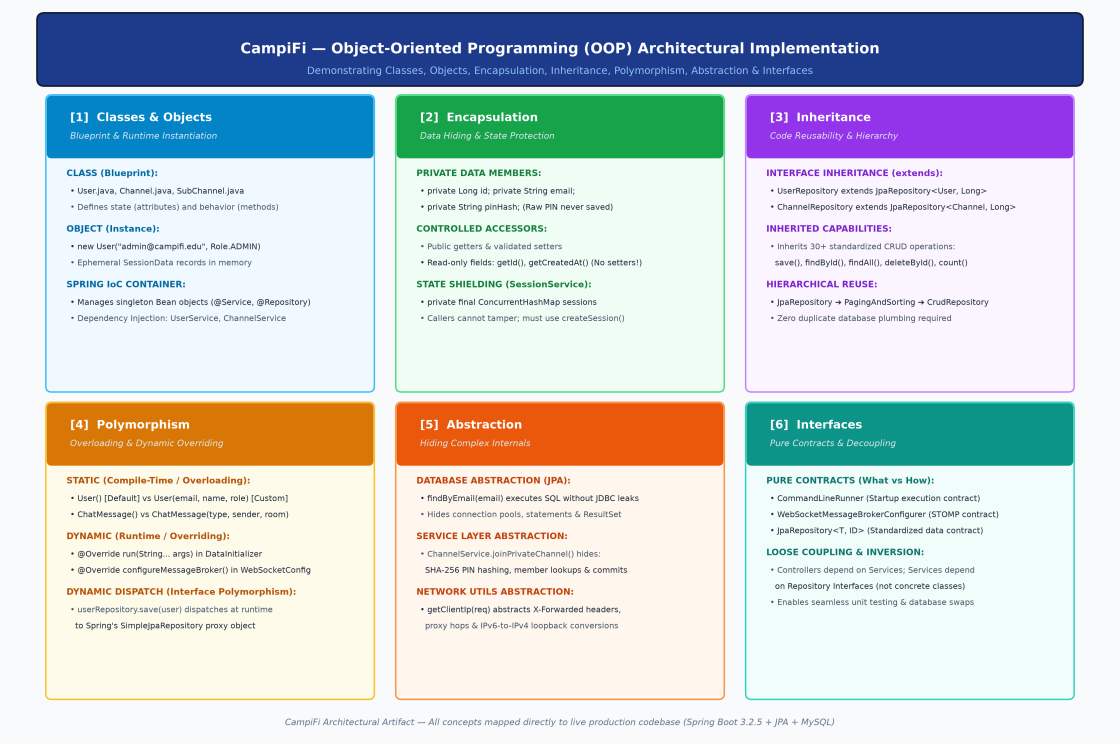


Figure 1: *Architectural Mapping of the 6 OOP Pillars across CampiFi's Domain Entities, Services, and Repositories.*

OOP Concept	Core Purpose in CampiFi	Concrete Java File & Reference
1. Classes & Objects	Blueprints defining domain state & dynamic runtime memory allocations	`User.java, Channel.java, SessionData`
2. Encapsulation	Data hiding via private members, controlled getters, and immutable IDs	`User.java (no ID setter), SessionService.java`
3. Inheritance	Code reusability via generic interface inheritance (30+ CRUD ops)	`UserRepository extends JpaRepository<User, Long>`
4. Polymorphism	Compile-time constructor overloading & runtime interface method overriding	`WebSocketConfig.java, DataInitializer.java`
5. Abstraction	Hiding SQL statements, network headers & hashing behind clean APIs	`ChannelService.java, NetworkUtils.java`
6. Interfaces	Pure contracts enabling loose coupling and Dependency Injection (IoC)	`CommandLineRunner, WebSocketMessageBroker...`

1. Classes and Objects: The Building Blocks of CampiFi

Definition: A **Class** is a programmer-defined blueprint that specifies the structure, attributes (state), and behaviors (methods) of a concept. An **Object** is a concrete instance of that class instantiated in memory (RAM) at runtime, holding actual state values.

A. Entity Class Blueprint: User.java

In CampiFi, User.java serves as the class blueprint representing participants on the offline hotspot. It defines persistent attributes mapped to the MySQL database via Jakarta Persistence annotations:

Code Reference: src/main/java/com/hotspotchat/model/User.java

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 255)
    private String email;

    @Column(nullable = false, length = 100)
    private String displayName;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role = Role.USER;

    @Column(nullable = false)
    private LocalDateTime createdAt = LocalDateTime.now();

    public User() {} // Required by JPA reflection
    public User(String email, String displayName, Role role) {
        this.email = email;
        this.displayName = displayName;
        this.role = role;
    }
    // Getters and Setters mediate access
}
```

B. Object Instantiation in Memory

Objects are actively created at runtime in two distinct ways throughout the application lifecycle:

- **Explicit Manual Instantiation (Heap Memory):** Using the new keyword to create domain entities. For example, in DataInitializer.java, startup objects are instantiated and populated into the database:

```
User admin = new User("admin@campifi.edu", "Master Admin", User.Role.ADMIN);
SubChannel sub = new SubChannel("Prof. Sharma", "Data Structures", parentChannel);
```

- **Spring IoC Managed Objects (Spring Beans):** The Spring ApplicationContext manages singleton lifecycle instances of classes annotated with @Service, @Repository, and @RestController. Classes such as UserService, ChannelService, and SessionService are instantiated once by Spring's Inversion of Control (IoC) container and injected where needed via Dependency Injection.

C. Modern Java 17 Records as Immutable Objects: SessionData

Inside SessionService.java, CampiFi uses a modern Java 17 **record** as an immutable data carrier object:

Code Reference: src/main/java/com/hotspotchat/service/SessionService.java (Lines 76-82)

```
public record SessionData(
    Long userId,
    String email,
    String displayName,
    User.Role role,
    LocalDateTime expiresAt
) {}
```

Examiner / Viva Question: What is the difference between a class and an object in CampiFi, and who instantiates the Service classes?

Ideal Presentation Answer: A class (e.g., User.java) is the compile-time blueprint defining fields and methods. An object is the runtime instance occupying RAM (e.g., an individual student logged in). While entity objects like User and Channel are created using the 'new' operator as data arrives, Service classes (UserService, ChannelService) are singleton Beans instantiated and managed automatically by the Spring Inversion of Control (IoC) Container.

2. Encapsulation: Information Hiding and State Protection

Definition: Encapsulation is the principle of bundling the internal data (fields) and the code (methods) that operates on that data into a single protective unit, while strictly restricting direct external access to internal state. Encapsulation guarantees data integrity, prevents unintended side effects, and allows internal logic to change without breaking calling code.

A. Defensive Field Access & The 'No-Setter' Principle

In User.java, all instance variables are declared private. Outside classes cannot directly read or mutate fields (e.g., user.email = "fake" produces a compiler error). Access is mediated exclusively through public getter and setter methods:

Code Reference: Encapsulation in User.java

```
public class User {
    private Long id;                  // Primary Key — managed by MySQL
    private String email;             // Institution email
    private LocalDateTime createdAt;  // Audit timestamp

    // Controlled Access: Getters allow external reading
    public Long getId() { return id; }
    public String getEmail() { return email; }
    public LocalDateTime getCreatedAt() { return createdAt; }

    // Validated Modification: Only email can be updated
    public void setEmail(String email) { this.email = email; }

    // CRITICAL ENCAPSULATION DESIGN:
    // Notice: There is NO public setId(Long id) or setCreatedAt(...)!
    // Preventing external classes from corrupting DB keys or history.
}
```

B. Sensitive State Protection: Channel PIN Hashing

Encapsulation protects sensitive security credentials in Channel.java and ChannelService.java. The raw 4-digit numeric PIN is never exposed, logged, or saved to the database. Instead, internal business logic encapsulates cryptographic hashing behind service boundaries:

Code Reference: State Protection & Cryptographic Encapsulation

```
// In Channel.java: Encapsulated private field
private String pinHash; // Stores 64-character SHA-256 hex string

// In ChannelService.java: Business logic encapsulation
if (type == Channel.ChannelType.PRIVATE && pin != null && !pin.isBlank()) {
    ch.setPinHash(hashPin(pin.trim())); // Encapsulated SHA-256 routine
}

private String hashPin(String pin) {
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    byte[] hash = digest.digest(pin.getBytes(StandardCharsets.UTF_8));
    return bytesToHex(hash); // Internal helper remains private!
}
```

C. Shielding Concurrency State in SessionService.java

In SessionService.java, the internal active session table is a private final ConcurrentHashMap. No outside controller can directly access, iterate over, or wipe the map. All operations must flow through encapsulated public API methods: createSession(), getSession(), and invalidate().

Examiner / Viva Question: Why did you deliberately omit setter methods for 'id' and 'createdAt' in your User entity class?

Ideal Presentation Answer: To enforce strict encapsulation and protect data integrity. The 'id' is a database surrogate primary key generated automatically by MySQL (IDENTITY). Allowing external code to invoke setId() could cause primary key collisions or hijack existing records. Similarly, 'createdAt' represents an immutable audit timestamp recording when the account was registered; omitting its setter guarantees audit trail tamper-resistance.

3. Inheritance: Code Reusability & Architectural Hierarchy

Definition: Inheritance is the mechanism by which one class or interface acquires the properties and behaviors (methods) of another class or interface using the extends keyword. Inheritance promotes massive code reuse, enforces structural consistency, and establishes an 'IS-A' relationship.

A. Generic Interface Inheritance: JpaRepository

In CampiFi, all persistence repositories implement **Interface Inheritance** by extending Spring Data's JpaRepository<T, ID> interface. Look at how UserRepository.java is declared:

Code Reference: src/main/java/com/hotspotchat/repository/UserRepository.java

```
@Repository
public interface UserRepository extends JpaRepository {
    // Custom query method declared via Spring Data derived naming
    Optional findByEmail(String email);
    boolean existsByEmail(String email);
}
```

B. The Multi-Level Inheritance Hierarchy in Spring Data

By extending JpaRepository<User, Long>, the UserRepository interface inherits from an extensive hierarchy of standard Java interfaces without writing a single line of redundant code:

1. **Repository<T, ID>** — Marker interface indicating a data access component.
2. **CrudRepository<T, ID>** (extends Repository) — Supplies core CRUD operations: save(), findById(), existsById(), findAll(), deleteById(), count().
3. **PagingAndSortingRepository<T, ID>** (extends CrudRepository) — Adds paginated and sorted querying methods: findAll(Sort), findAll(Pageable).
4. **ListPagingAndSortingRepository<T, ID>** (extends PagingAndSortingRepository) — Refines return types to java.util.List.
5. **JpaRepository<T, ID>** (extends ListPagingAndSortingRepository) — Adds JPA-specific batching and flush operations: flush(), saveAndFlush(), deleteAllInBatch().
6. **UserRepository** (extends JpaRepository) — Inherits all 30+ methods above for the User entity!

C. Inheritance across All 7 Domain Repositories

CampiFi reuses this identical inheritance structure across all entity components in the application:

Repository Interface	Parent Interface Extended	Inherited Generic Types & Key Inherited Methods
`UserRepository`	`JpaRepository<User, Long>`	Inherits save(User), findById(Long), deleteById(Long)
`ChannelRepository`	`JpaRepository<Channel, Long>`	Inherits count(), save(Channel), findAll()
`SubChannelRepository`	`JpaRepository<SubChannel, Long>`	Inherits save(SubChannel), findById(Long)
`MessageRepository`	`JpaRepository<Message, Long>`	Inherits batching, pagination, and persistence operations

Examiner / Viva Question: Does Java support multiple inheritance of classes? How does your project achieve multiple inheritance?

Ideal Presentation Answer: Java does NOT support multiple inheritance of classes (to prevent the famous 'Diamond Problem' of conflicting method implementations). However, Java explicitly supports MULTIPLE INHERITANCE OF INTERFACES using the 'implements' keyword or by having an interface 'extend' multiple parent interfaces. In CampiFi, JpaRepository inherits from both ListCrudRepository and ListPagingAndSortingRepository, cleanly combining multiple behavioral contracts without method conflicts.

4. Polymorphism: Compile-Time Overloading & Runtime Dynamic Overriding

Definition: Polymorphism (Greek for 'many forms') allows a single interface, method, or object to execute different behaviors depending on how it is invoked. Java implements polymorphism in two primary flavors: **Compile-time (Static) Polymorphism** via Method/Constructor Overloading, and **Runtime (Dynamic) Polymorphism** via Method Overriding and Interface Dynamic Dispatch.

A. Compile-Time Polymorphism: Constructor Overloading

In User.java, multiple constructors exist with the same name but distinct parameter signatures. The Java compiler determines which constructor to invoke at compile time based on the argument list:

Code Reference: Constructor Overloading in User.java

```
public class User {
    // Form 1: Default No-Argument Constructor (Required by Hibernate JPA via reflection)
    public User() {}

    // Form 2: Parameterized Constructor (Used by application code during user registration)
    public User(String email, String displayName, Role role) {
        this.email = email;
        this.displayName = displayName;
        this.role = role;
    }
}
```

B. Runtime Polymorphism: Method Overriding (@Override)

In **Runtime Polymorphism**, a class implements an interface method, providing its own custom behavior. The Java Virtual Machine (JVM) determines which implementation to execute dynamically at runtime. Two prominent examples in CampiFi are:

Code Reference: Dynamic Method Overriding in WebSocketConfig & DataInitializer

```
// Example 1: Overriding WebSocket Broker Configuration
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws").setAllowedOriginPatterns("*").withSockJS();
    }
}

// Example 2: Overriding Spring Boot Startup Hook
@Component
public class DataInitializer implements CommandLineRunner {
    @Override
    public void run(String... args) {
        // Custom startup seeding logic executed dynamically by Spring Boot
        if (channelRepository.count() > 0) return;
        seedDefaultCampusNetworks();
    }
}
```

C. Dynamic Dispatch via Interface References

In ChannelService.java, dependencies are declared using **interface types** rather than concrete classes:

```
private final UserRepository userRepository; // Interface reference type
```

When userRepository.save(user) is called, the JVM does not execute code inside an interface (which has none). Instead, it dynamically dispatches the call to the runtime proxy instance generated by Spring (SimpleJpaRepository). This dynamic polymorphism enables loose coupling and makes swapping implementations seamless.

Examiner / Viva Question: What is the difference between Method Overloading and Method Overriding in CampiFi?

Ideal Presentation Answer: Method Overloading (Compile-Time / Static) occurs when two methods in the same class share the same name but have different parameter lists (e.g., User() vs User(email, name, role)); the compiler resolves which method to invoke. Method Overriding (Runtime / Dynamic) occurs when a subclass or implementing class provides its own specific implementation of an inherited or interface method using the @Override annotation (e.g., WebSocketConfig overriding configureMessageBroker()); the JVM resolves the exact method at runtime via dynamic dispatch.

5. Abstraction: Hiding Complexity and Exposing Essentials

Definition: Abstraction is the principle of hiding complex, low-level implementation details and exposing only the essential, high-level features and operations to the caller. Abstraction allows developers to work with intuitive domain operations without getting bogged down by low-level networking, database protocols, or memory buffers.

A. Persistence Abstraction via Spring Data JPA

Consider this single line in UserRepository.java:

```
Optional<User> findByEmail(String email);
```

This concise method declaration is a masterclass in abstraction. Behind this single method call, Spring Data JPA and Hibernate abstract away dozens of low-level procedural steps:

1. Opening a raw TCP socket connection to MySQL on port 3306 via the HikariCP connection pool.
2. Preparing the parameterized SQL statement: SELECT * FROM users WHERE email = ? LIMIT 1.
3. Binding the Java String parameter to prevent SQL injection vulnerabilities.
4. Executing the query across the network and reading the raw byte stream from MySQL.

5. Parsing the JDBC ResultSet rows and columns into Java primitive types.
6. Instantiating a new User entity object and populating its fields via Java reflection.
7. Wrapping the result in a modern Java Optional<User> container to eliminate null pointer risks.

The service layer simply invokes userRepository.findByEmail(email) without ever knowing raw SQL exists!

B. Service Layer Business Abstraction: ChannelService

Controllers never directly query the database or perform raw cryptographic operations. Instead, ChannelService.java provides high-level business abstraction methods:

Code Reference: Business Logic Abstraction in ChannelService.java

```
@Transactional
public Channel createChannel(String name, String description, Channel.ChannelType type, String pin) {
    if (channelRepository.existsByName(name.trim())) {
        throw new IllegalArgumentException("Channel name already exists.");
    }
    Channel ch = new Channel();
    ch.setName(name.trim());
    ch.setDescription(description);
    ch.setType(type);
    if (type == Channel.ChannelType.PRIVATE && pin != null && !pin.isBlank()) {
        ch.setPinHash(hashPin(pin.trim())); // PIN hashing abstracted!
    }
    return channelRepository.save(ch); // Transaction boundary abstracted!
}
```

C. Network Hardware & IP Abstraction: NetworkUtils.java

Handling client IP addresses on an offline Wi-Fi hotspot involves handling reverse proxies, forwarded headers, and IPv6 loopback edge cases. NetworkUtils.java completely abstracts this complexity:

Code Reference: Hardware & Socket Abstraction in NetworkUtils.java

```
@Component
public class NetworkUtils {
    public String getClientIp(HttpServletRequest request) {
        String ip = request.getHeader("X-Forwarded-For");
        if (ip != null && !ip.isEmpty() && !"unknown".equalsIgnoreCase(ip)) {
            ip = ip.split(",")[0].trim(); // Extract first proxy hop
        } else {
            ip = request.getHeader("X-Real-IP");
        }
        if (ip == null || ip.isEmpty() || "unknown".equalsIgnoreCase(ip)) {
            ip = request.getRemoteAddr(); // Direct socket connection
        }
        if ("::1".equals(ip) || "0:0:0:0:0:0:0:1".equals(ip)) ip = "127.0.0.1";
        return ip; // Caller receives a clean, normalized IPv4 address!
    }
}
```

Examiner / Viva Question: How does Spring Data JPA demonstrate abstraction in your persistence layer?

Ideal Presentation Answer: Spring Data JPA abstracts the entire database interaction. In UserRepository, we write 'Optional findByEmail(String email);' without any SQL or JDBC code. Spring automatically abstracts away connection pool management, parameterized SQL generation, statement execution, JDBC ResultSet parsing, and object-relational mapping. The caller works strictly with domain objects, completely shielded from low-level database mechanics.

6. Interfaces: Pure Contracts and Architectural Decoupling

Definition: An **Interface** in Java is a completely abstract type that acts as a pure contract, specifying *what* behaviors a class must implement without defining *how* they must be implemented. Interfaces enable loose coupling, modularity, multiple inheritance of type, and adherence to the Dependency Inversion Principle (DIP).

A. Core Interfaces Used Across CampiFi

CampiFi leverages three fundamental interface contracts to structure its architecture:

- **CommandLineRunner** (Spring Framework): A functional interface containing exactly one contract method: void run(String... args). Any Spring Bean implementing this interface is guaranteed to execute immediately after the application context loads. Used in DataInitializer.java to automatically seed campus channels.
- **WebSocketMessageBrokerConfigurer** (Spring Messaging): An interface defining callback hooks for configuring STOMP endpoints and real-time message routing. Used in WebSocketConfig.java.
- **JpaRepository<T, ID>** (Spring Data): A generic contract defining standardized database operations. Implemented by all 7 repository interfaces in CampiFi.

B. The Dependency Inversion Principle (DIP) in Action

One of the most important software engineering reasons for using interfaces is the **Dependency Inversion Principle** (the 'D' in SOLID): *High-level modules should not depend on low-level modules; both should depend on abstractions (interfaces).*

Code Reference: Dependency Inversion in ChannelService.java

```
@Service
public class ChannelService {
    // DEPENDENCY INVERSION: The service depends on the INTERFACE,
    // NOT on a concrete database implementation class!
    private final ChannelRepository channelRepository;
    private final UserRepository    userRepository;

    @Autowired
    public ChannelService(ChannelRepository channelRepository,
                          UserRepository userRepository) {
        this.channelRepository = channelRepository;
        this.userRepository    = userRepository;
    }
}
```

Why this interface-driven architecture is critical for enterprise software:

1. **Loose Coupling:** ChannelService does not know or care whether the repository is backed by MySQL, PostgreSQL, or an in-memory H2 database.
2. **Unit Testing with Mockito:** In unit tests, developers can easily inject a mock repository interface (e.g., Mockito.mock(UserRepository.class)) to test business logic in isolation without requiring a live MySQL server running.
3. **Zero Boilerplate:** Interfaces eliminate thousands of lines of handwritten SQL and JDBC plumbing code.

Examiner / Viva Question: Why did you write UserRepository as an interface instead of a regular class?

Ideal Presentation Answer: Because UserRepository follows the interface contract pattern of Spring Data JPA. By declaring an interface that extends JpaRepository, we specify WHAT database operations are needed, not HOW to implement them. At runtime, Spring dynamically generates a proxy class implementing the interface. Furthermore, depending on an interface adheres to the Dependency Inversion Principle (SOLID), decoupling our business services from database specifics and enabling effortless unit testing with mock objects.

7. OOP Master Matrix & Viva Examination Defense

Use this master reference matrix during your project evaluation to immediately point examiners to the exact file, line number, and technical justification for each OOP concept:

OOP Concept	Code File & Exact Location	Implementation Details in CampiFi	Examiner Viva Talking Point
Classes & Objects	`model/User.java` `service/SessionService.java`	Domain entity blueprint; SessionData record; runtime instantiation via new & Spring Beans	Separation of compile-time structure (class) from runtime heap state (object).
Encapsulation	`model/User.java` `model/Channel.java`	Private instance variables, public getters/setters, deliberate omission of setId() & setCreatedAt()	Protects database keys and audit timestamps from accidental tampering.
Inheritance	`repository/UserRepository.java` `repository/ChannelRepository.java`	extends JpaRepository<T, ID> inheriting 30+ CRUD and paging methods without duplication	Multi-level interface inheritance hierarchy (Repository → CrudRepository → JpaRepository).
Polymorphism (Static)	`model/User.java` `model/ChatMessage.java`	Constructor overloading: User() (JPA reflection) vs User(email, name, role) (custom)	Compile-time polymorphism resolved by the compiler via parameter signature matching.
Polymorphism (Dynamic)	`config/WebSocketConfig.java` `DataInitializer.java`	Method overriding via @Override configureMessageBroker() and @Override run()	Runtime polymorphism resolved dynamically by the JVM via dynamic dispatch.
Abstraction	`repository/UserRepository.java` `service/NetworkUtils.java`	findByEmail() hides raw SQL, JDBC & socket pools; getClientIp() hides HTTP proxies	Hides low-level complexity behind clean, intuitive domain interfaces.
Interfaces	`CommandLineRunner` `WebSocketMessageBroker...`	Pure contracts implemented by service components; enables Dependency Inversion (DIP)	Enables loose coupling and seamless unit testing with mock objects.

Top 4 'Golden Rules' to Remember for Your Project Defense

1. **Always pair the concept with a specific file:** Never just say 'we used encapsulation'—say *'We implemented encapsulation in User.java by declaring all fields private and deliberately omitting setId() to prevent database key tampering.'*
2. **Distinguish Compile-Time from Runtime:** When asked about polymorphism, immediately clarify that CampiFi uses BOTH: constructor overloading at compile time, and interface method overriding (WebSocketConfig) at runtime.
3. **Emphasize Abstraction vs Encapsulation:** *Encapsulation* is about **data protection** (hiding state inside a class). *Abstraction* is about **complexity reduction** (hiding how a feature is implemented behind a simple method like findByEmail).
4. **Highlight the Dependency Inversion Principle:** Show that your service classes inject repository *interfaces*, not concrete classes, proving clean architectural decoupling.